

# DSE6212

## Text and Image Mining Session # 3

---

Agarwal



MERRIMACK COLLEGE

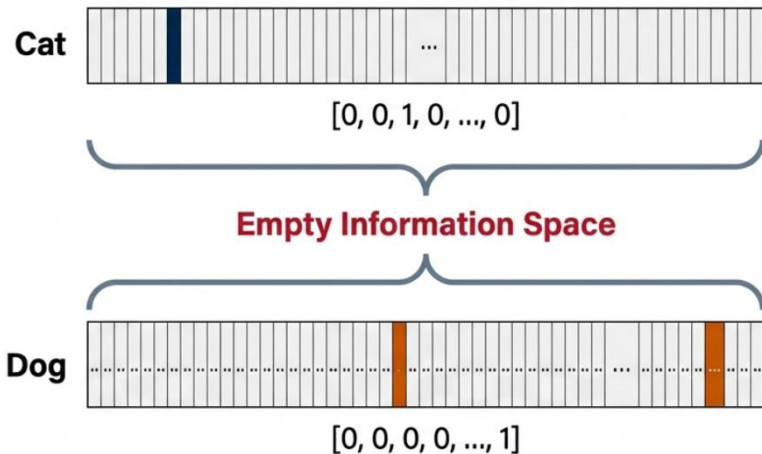
# Word Embeddings

# Limitations of Sparse Representation

Classical NLP treats words as **atomic units** using discrete indices in a vocabulary as exemplified by [One-Hot Encoding & Bag of Words](#). As visualized in the 'Dog vs. Cat' orthogonality examples, this approach is fundamentally limited:

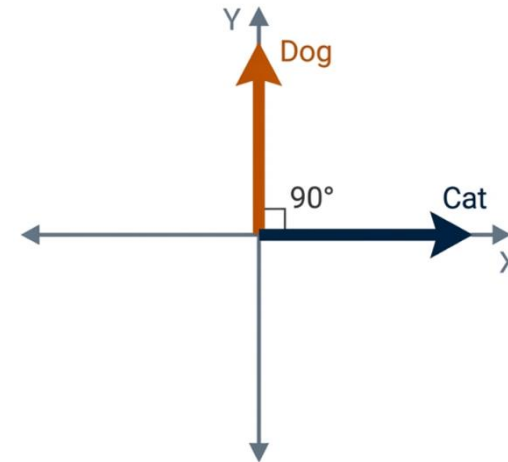
## Sparsity & High Dimensionality:

Vector size scales linearly with vocabulary  $|V|$ , creating massive, computationally inefficient sparse matrices



## Semantic Blindness:

In a one-hot space, the dot product of any two distinct words is 0. Geometrically, "dog" and "cat" are as unrelated as 'dog' and 'refrigerator' because they are perfectly orthogonal



**Word Embeddings** resolve this by mapping words to **dense, low-dimensional vectors** in  $R^d$  ( $d \ll |V|$ ). Grounded in the [Distributional Hypothesis](#), these representations ensure that words appearing in similar contexts are positioned near one another in the vector space.

# Distributional Hypothesis

The **Distributional Hypothesis** is the foundational theoretical pillar of modern word embeddings, famously encapsulated by linguist J.R. Firth's 1957 dictum:

*"You shall know a word by the company it keeps."*

In the context of Machine Learning, it posits that "words that occur in similar contexts tend to have similar meanings"

Government debt problems  
turning into ... crises.

**BANKING**

Europe's central  
... system.

Catering to the  
corporate ... sector.

# Distributional Hypothesis

**Contextual Proximity:** In a scientific corpus using example such as

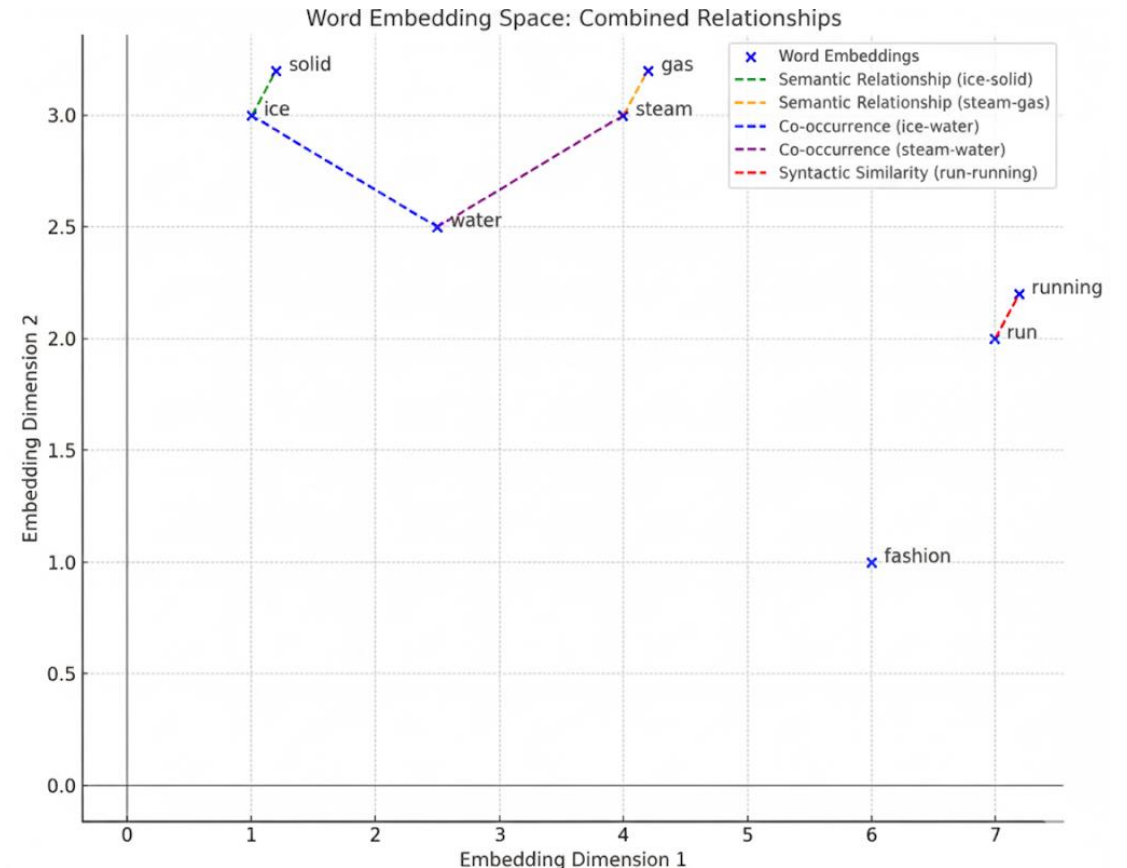
("Water freezes into solid ice", "Water boils into steam")

the word "ice" appears near "solid," and "steam" near "gas." The model clusters this pairs in the vector space.

**Bridging Relationships:** The word "water" co-occurs with both "ice" (freezing water) and "steam" (boiling water), it acts as a **semantic bridge** in the geometry

**Discriminative Geometry:** Unrelated words like "fashion" lack these shared context clues and are thus placed in distant regions.

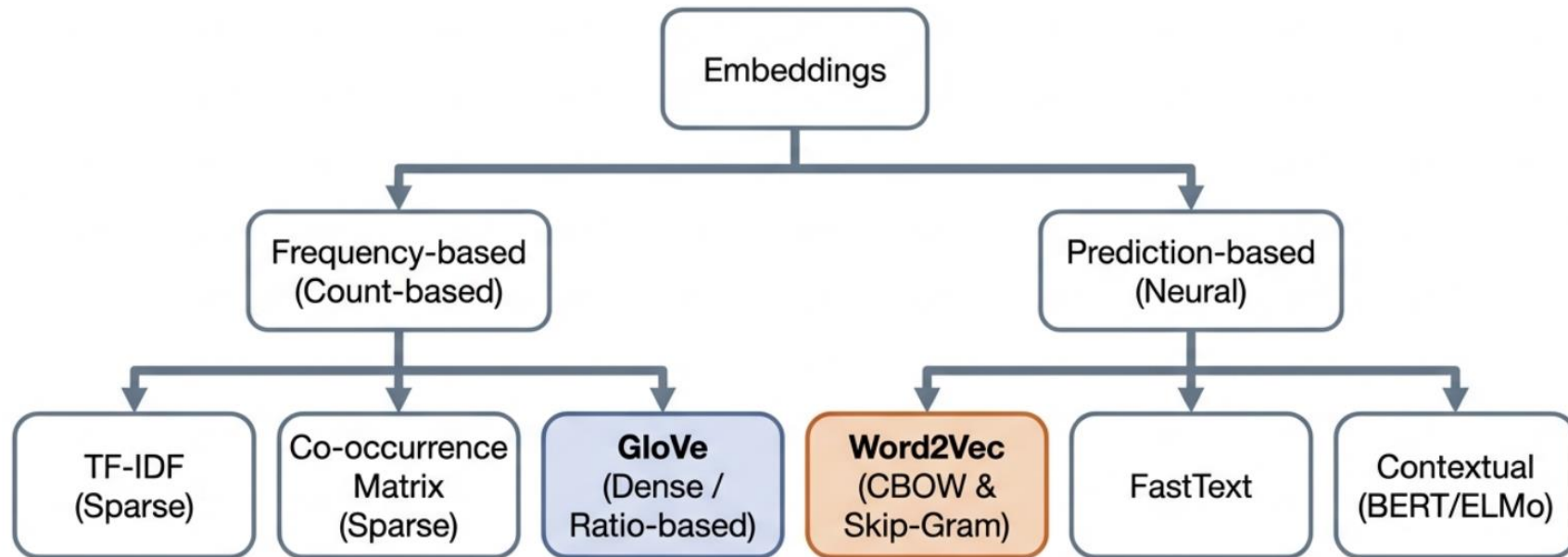
**Syntactic vs. Semantic:** While "ice" and "steam" are linked by their shared chemistry (semantic), words like "run" and "running" are placed together due to their shared grammatical roots (syntactic), yet they remain distant from the water-related cluster because their surrounding context is entirely different.



# Word2vec

Introduced by Mikolov et al. (2013), Word2Vec shifted the focus from counting co-occurrences to **predicting** them using shallow neural networks. It primarily consists of two architectures:

- Continuous Bag-of-Words (CBOW)
- Skip-Gram





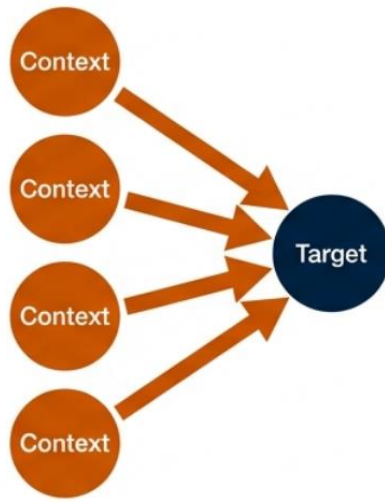
# Word2vec: Continuous bag-of-words (CBOW)

CBOW: Is a neural network-based model that learns to predict a **target** word ( $w_T$ ) based on its surrounding **context** words.

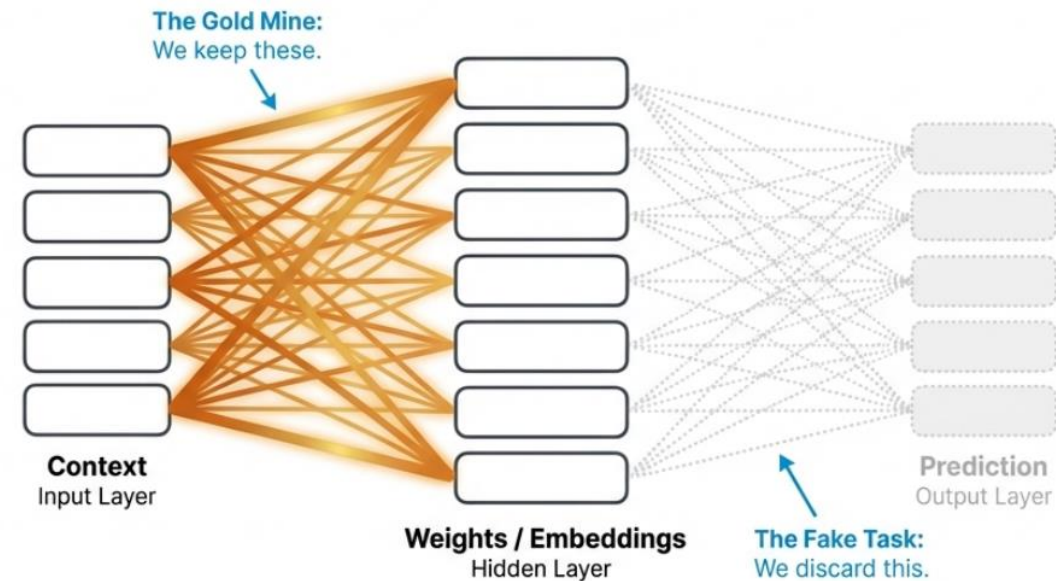
*“The quick brown **fox** jumps over the lazy dog”*

**Intuition:** We are training a neural network to solve a task we don't care about – predicting a missing word – to acquire a byproduct we need: the internal weight matrices

## CBOW (Continuous Bag of Words)



Predicts Target from Context.  
Faster training.



# Word2vec: Continuous bag-of-words (CBOW)

## The Training Setup: Sliding Windows

To generate training samples, we slide a fixed-size window over the raw text corpus. This creates pairs of **(Context, Target)**.

### Example:



### Training Data;

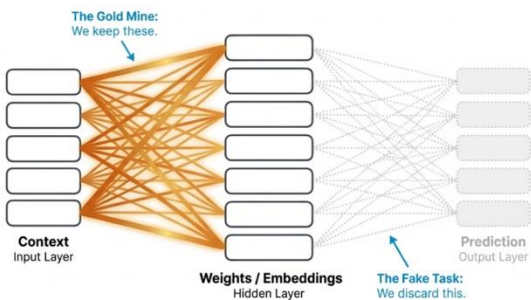
| Input                             | Output  |
|-----------------------------------|---------|
| "quick", "brown", "jumps", "over" | "fox"   |
| "brown", "fox", "over", "lazy"    | "jumps" |
| "fox", "jumps", "lazy", "dog"     | "over"  |



# CBOW Step by Step Process

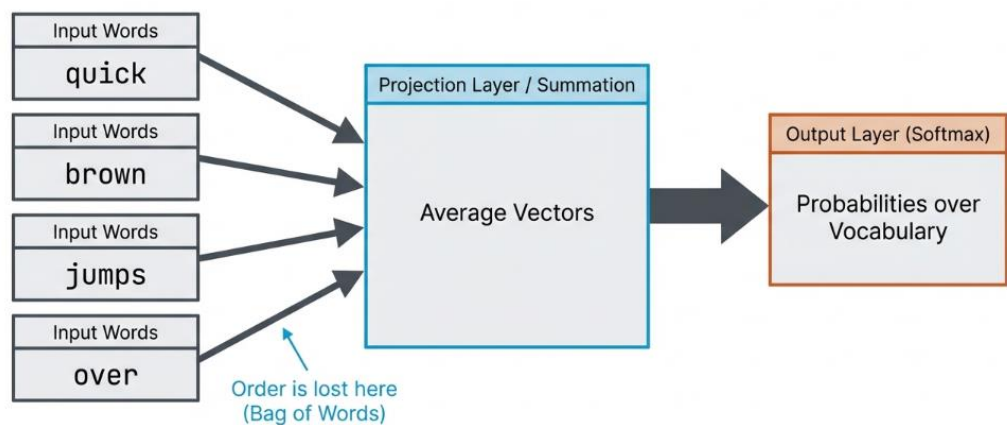
## Step 1: Input Layer (One-Hot Encoding)

Each word in the context window is initially represented as a **One-Hot Vector** of size  $V$  (the total vocabulary size). In this vector, only the index corresponding to that specific word is set to **1**, while all others are **0**

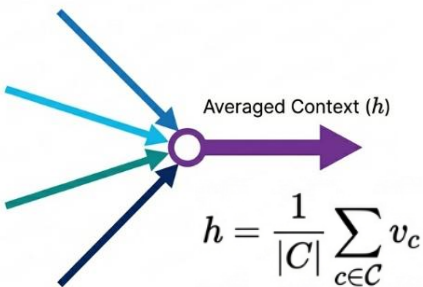
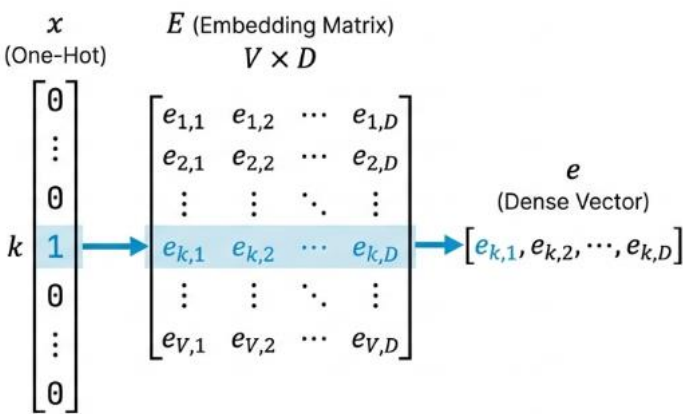


## Step 2: The Projection Layer (Dense Lookup)

We multiply the one-hot vectors by the **Embedding Matrix ( $E$ )** of size  $V \times D$ . This transforms the sparse, high-dimensional input into a dense vector ( $e$ ) of dimension  $D$ .



|         |                       |         |                       |
|---------|-----------------------|---------|-----------------------|
| "quick" | [1, 0, 0, 0, 0, 0, 0] | "fox"   | [0, 0, 1, 0, 0, 0, 0] |
| "brown" | [0, 1, 0, 0, 0, 0, 0] | "jumps" | [0, 0, 0, 1, 0, 0, 0] |



## Step 3: The Hidden Layer (Averaging the Context)

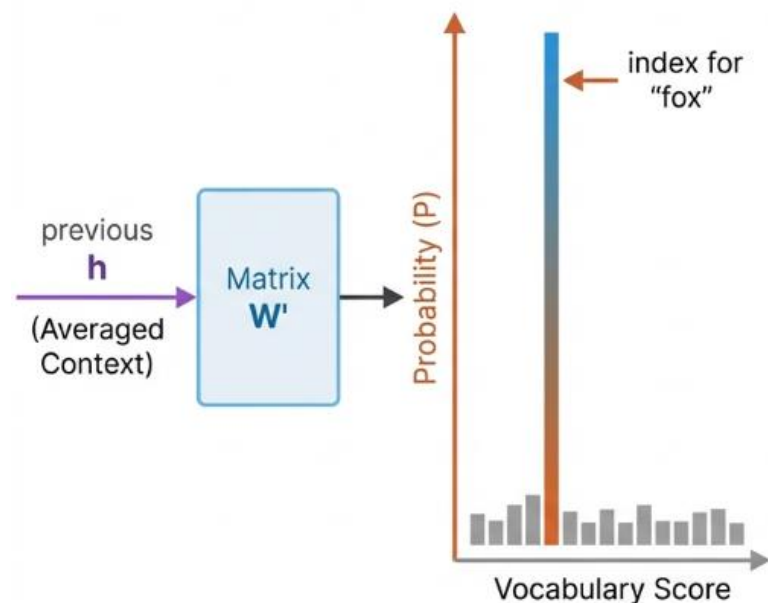
The model averages the context vector to create a single summary vector  $h$

Note: This summation compresses information and reduces noise, but it destroys word order, effectively creating **The Bag-of-Words Effect**

# CBOW Step by Step Process

## Step 4: The Output Layer (Generating Probabilities)

The averaged context vector ( $h$ ) is multiplied by a second weight matrix to produce a score for every word in the vocabulary. Finally, a Softmax function converts these raw scores (logits) into a probability distribution summing to 1. The model is trained to maximize the probability of the target word given its context words. During training, the model adjusts the weights and biases of the neurons to minimize the error between the predicted target word embedding and the true target word embedding.



$$P(w_t | context) = \frac{\exp(v_{w_t} \cdot h)}{\sum_{w' \in V} \exp(v_{w'} \cdot h)}$$

### The Input: Dot Product Similarity ( $v_{w_t} \cdot h$ )

The term in the numerator, represents the similarity score (or logit) for the true target word.

$h$ (Averaged Context): This is the vector generated in the Hidden Layer. It is the mathematical "summary" of all the words surrounding the target.

$v_{w_t}$ (Output Vector): This is the candidate vector for the target word (e.g., "fox") stored in the output weight matrix.

The Operation: The dot product measures how well the averaged context aligns with the target word. A higher dot product means the model believes that the context "quick, brown, jumps, over" is a strong match for the word "fox."

A key advantage of CBOW is its ability to learn these word embeddings in an unsupervised manner, meaning it doesn't require labeled data. This allows CBOW to leverage vast amounts of readily available unlabeled text data for training.

# Word2vec: CBOW

- **Advantages:**

- Computational Efficiency: CBOW treats the entire context window as a single observation, making it faster to train, especially for large datasets, as it learns by predicting a single word from multiple context words.
- Frequent Word Mastery: Because the model averages context, it effectively "smooths out" noise, resulting in very robust representations for high-frequency terms.
- Fast Convergence: The simplified architecture allows for rapid learning of broad semantic categories

- **Limitations:**

- Rare Word Weakness: The signal from a rare word in a context window is often "drowned out" by the average of more frequent neighbors.
- Positional Loss: By treating context as a "bag," the model fails on tasks requiring strict word order or syntactic precision.
- Ambiguity: It struggles to distinguish between multiple meanings of a word if their typical context windows overlap significantly.

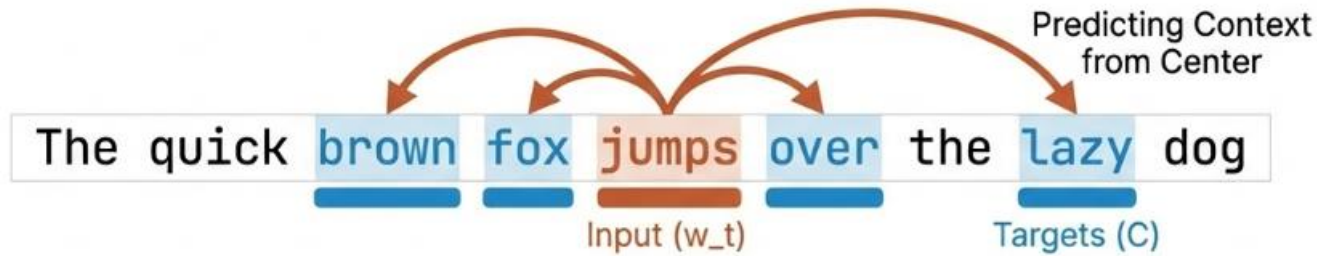
- **Practical Use Cases of CBOW**

- Sentiment Analysis: Classifying text based on broad semantic cues where word order is less critical than general keyword presence.
- Spam Filtering: Identifying clusters of suspicious terms that tend to appear together across various contexts.
- Domain-Specific Embeddings: Generating specialized vectors for medicine or law where frequent technical terms dominate the semantics.

# Word2vec: Skip-Gram

**Skip-Gram** reverses CBOW logic, given a **target word**, it predicts **the surrounding context words** independently. It is generally superior for representing rare words

**Flipping the Prediction Task:**



Logic:

- **Input (Target):** "jumps"
- **Output (Context):** ["brown", "fox", "over", "lazy"]
- **Resulting Pairs:** (brown, fox), (fox, over), (over, lazy)

## Skip-Gram



Predicts Context from Target.  
Better for rare words.

# Word2vec: Skip-Gram

## Step-by-Step Process

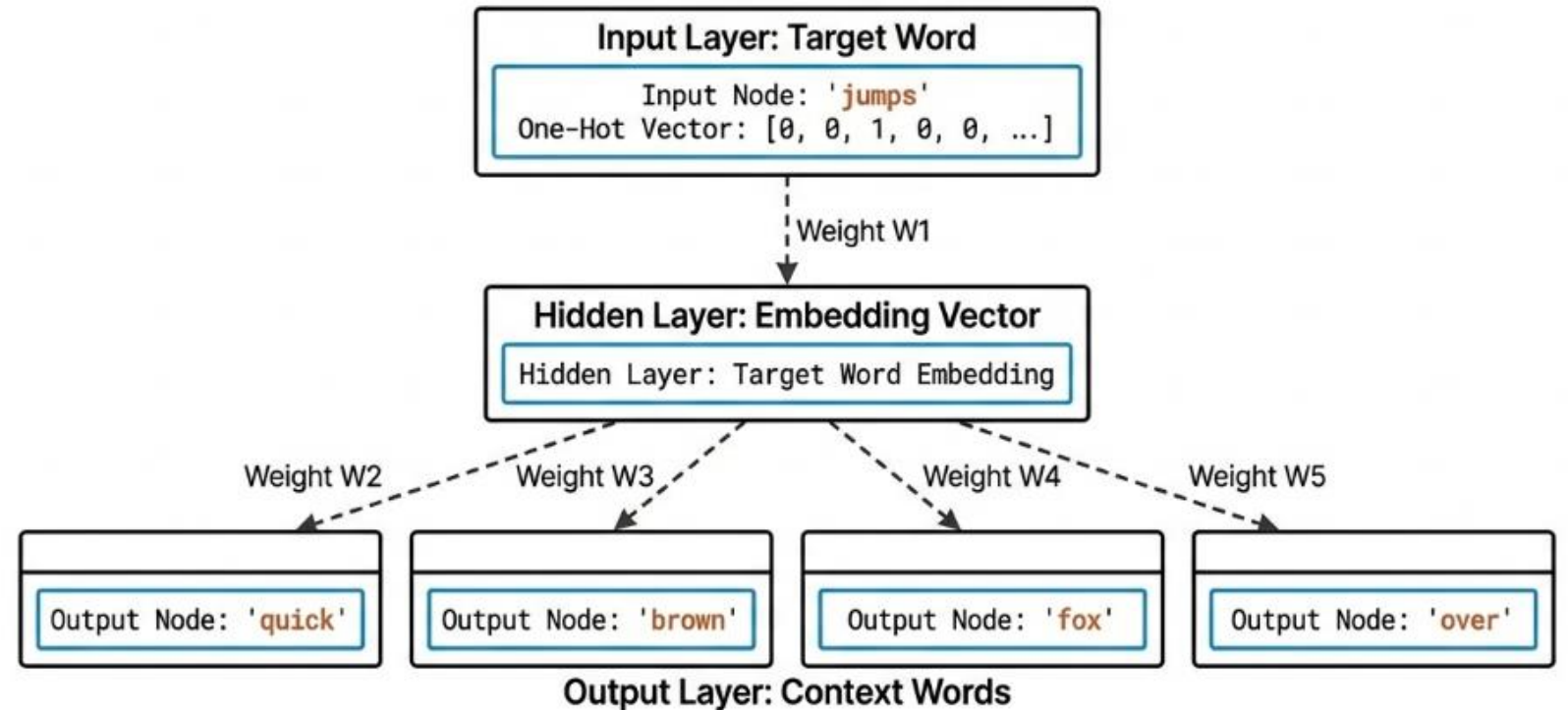
### Step 1: Single Word Projection (The Hidden Layer)

Because Skip-Gram uses only one input word at a time, there is **no averaging** in the hidden layer. The hidden state ( $h$ ) is simply the dense vector of the target word:  $h = v_{w_t}$

### Step 2: Multiple Context Predictions (The Output Layer)

The model treats each context word as an independent prediction task. For a target word and a context word, the probability is:

$$P(\text{context}|w_t) = \prod_{c \in \text{context}} \frac{\exp(v_c \cdot v_{w_t})}{\sum_{w' \in V} \exp(v_{w'} \cdot v_{w_t})}$$



This approach allows Skip-gram to capture the semantic relationships between words by analyzing their co-occurrence patterns in a large corpus of text. Effective for representing rare words by training on multiple contexts

# Word2vec: Skip-gram

## Advantages (Pros)

- **Rare Word Precision:** Because each target-context pair is a separate training update, rare words are not averaged away. They provide a distinct signal that the model can learn from.
- **Nuanced Representations:** It captures more detailed semantic and syntactic relationships than CBOW, especially in larger datasets.

## Limitations (Cons)

- **Underutilization of Global Statistics:** Word2vec scans local windows but ignores the vast repetition and global co-occurrence counts across the whole corpus
- **Redundancy Inefficiency:** Training scales linearly with corpus size  $|C|$ . It re-learns the ‘the cat’ millions of times individually rather than once as per statistics

## Practical Use Cases

- **Analogy Tasks:** Solving "Man is to King as Woman is to X."  $\text{Vector}(\text{King}) - \text{Vector}(\text{Man}) + \text{Vector}(\text{Woman}) \approx \text{Vector}(\text{Queen})$
- **Rare Term Discovery:** Specialized fields like medical research or niche technical documentation where rare terms carry critical meaning.



# Appendix

- CBOW Probability Formula:

$$P(w_t|w_{context}) = \frac{\exp(v_{w_t}^T \cdot h)}{\sum_{w \in V} \exp(v_w^T \cdot h)}$$

Where:

- $w_t$ : Target word
- $w_{context}$ : Context words
- $V_{w_t}$ : Embedding vector of the target word
- $h$ : Average of embedding vectors of context words
- $exp$ : Exponential function to ensure non-negative probabilities
- $V$ : Vocabulary size for normalization

- CBOW Loss function:

$$J(\theta) = -\log P(w_t|w_{context})$$

Where:

- $J(\theta)$ : Negative log-likelihood loss function
- $P(w_t|w_{context})$ : probability of the target word

- Skip-gram Probability Formula:

$$P(w_{context}|w_t) = \prod_{i=1}^N \frac{\exp(v_{w_i}^T \cdot h)}{\sum_{w \in V} \exp(v_w^T \cdot h)}$$

Where:

- $V_{w_t}$ : Embedding vector of the target word
- $V_{w_i}$ : Embedding vector of the i-th context word
- $V$ : Vocabulary size for normalization
- $\prod_{i=1}^N$  Aggregates the probabilities of all  $N$  context words

- Skip-gram Loss function:

$$J(\theta) = -\sum_{i=1}^N \log P(w_{context}|w_t)$$

The loss accumulates the negative log-likelihood for each context word independently

Requires separate predictions for every context word around the target word

# GloVe

## Global Vectors for Word Representation

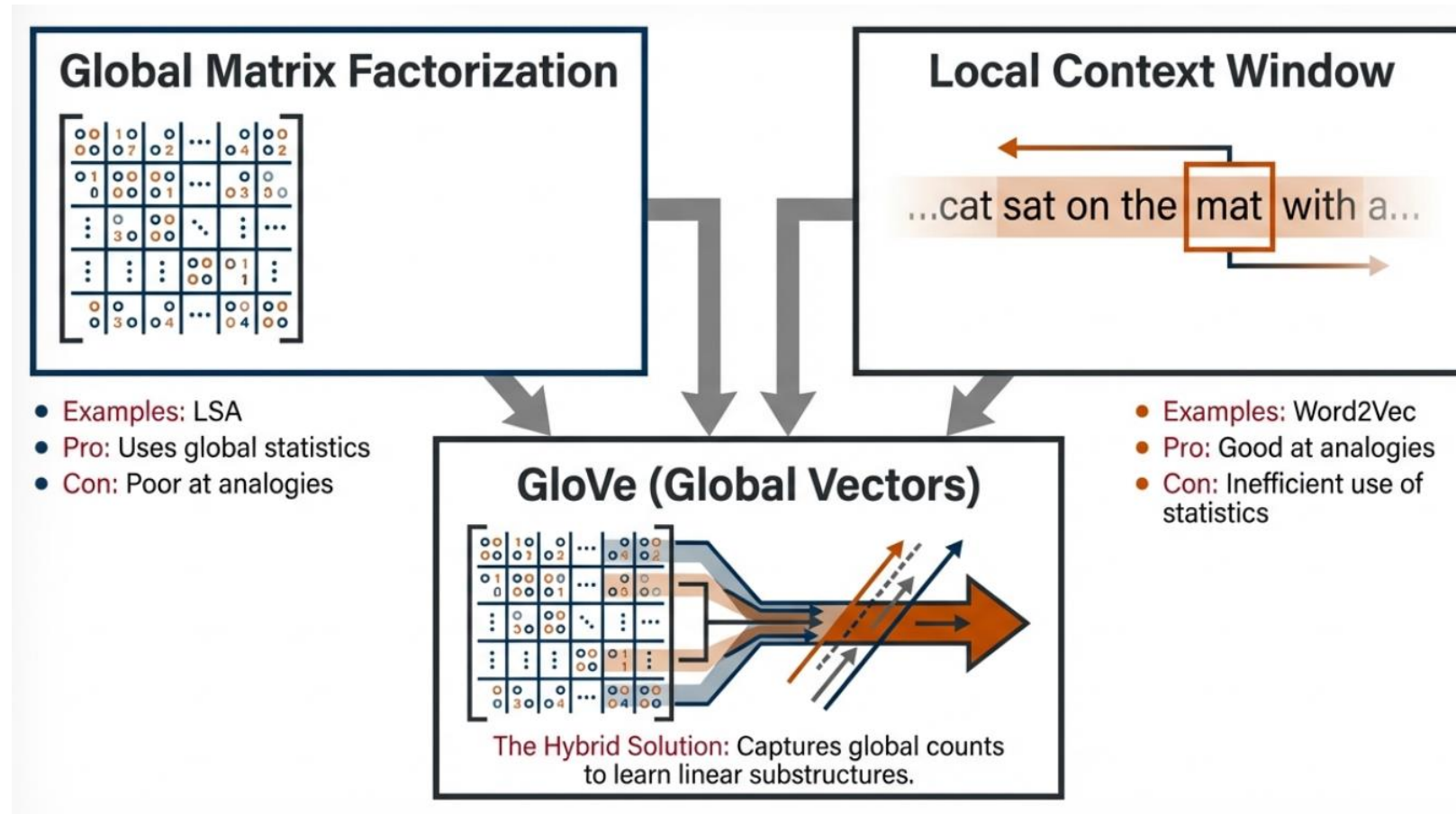
# GloVe: Global Vectors for Word Representation

While Word2vec revolutionized embeddings through local predictive modeling, it possesses inherent architectural limitations that led to the development of GloVe:

- **Underutilization of Global Statistics:** Word2vec scans local windows across the corpus. While effective for capturing local semantics, it fails to take advantage of the vast amount of repetition and global co-occurrence statistics in the data. *It sees "slices" of context rather than the "whole picture."*
- **Computational Inefficiency on Redundancy:** *Training scales linearly with the corpus size  $|C|$ .* In a massive dataset with high redundancy, Word2vec spends significant cycles re-learning the same local patterns without aggregating them into a global statistical profile.
- **Opaque Relationship to Ratios:** Word2vec produces linear substructures (analogies) as a byproduct, but the underlying reason *why* these emerge remains mathematically opaque within a purely local predictive framework (Word2vec is like a person who learns a language fluently by listening to neighbors but cannot explain the grammar rules. While it produces impressive results (like solving analogies), the mathematical reason *why* these relationships emerge is not explicitly built into the code; it is an unexplained "lucky byproduct" of the training process)

These limitations prompted Pennington et al. to design GloVe, a model that explicitly utilizes global word-word co-occurrence counts while preserving the linear regularities found in Word2vec.

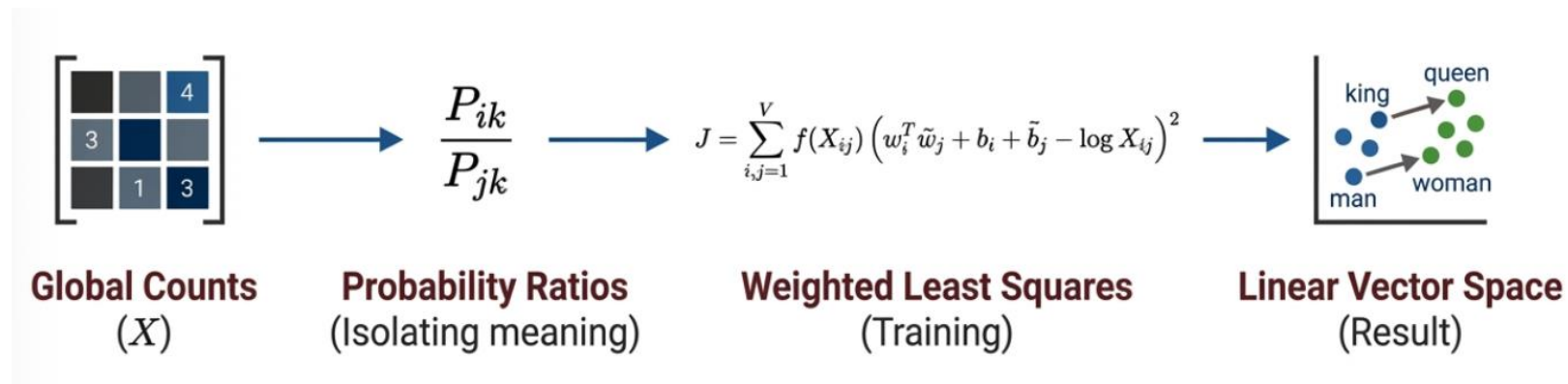
# GloVe: Global Vectors for Word Representation



GloVe (Pennington et al., 2014) is a **Hybrid Solution** that combines global statistical "counting" with local relational learning (Word2Vec)

# GloVe: How Global learns and Operates: Step by Step

GloVe (Pennington et al., 2014) is a **Hybrid Solution** that combines the statistical strengths of "counting" (Global Matrix Factorization) with the relational accuracy of "predicting" (Word2Vec).



The model transitions from raw data to a linear vector space through a structured four-step process:

## Step 1: Global Data Aggregation (Building Matrix X )

Instead of scanning text line-by-line during training, GloVe pre-calculates a global **word-word co-occurrence matrix (X)**.

- $X_{ij}$  : Stores how many times word  $j$  appears in the context of word  $i$
- **Distance Weighting**: It weights counts by  $1/d$  (where  $d$  is distance). Words immediately next to each other contribute more to the "meaning" than those further away.

# GloVe: How Global learns and Operates: Step by Step

## Step 1: Global Data Aggregation (Building Matrix $X$ ) - Continued

Instead of scanning text line-by-line during training, GloVe pre-calculates a global word-word co-occurrence matrix ( $X$ ).

- **Determining Distance ( $d$ ):** As the model scans the corpus, it uses a fixed window size (e.g., 10 words). The distance  $d$  is the number of tokens separating context word from target word.
  - For example, in "The cat sat on the mat,"
    - if "mat" is the target: "the" has ( $d$ ) = 1 and "on" has ( $d$ ) = 2
- **Distance Weighting ( $1/(d)$ ):** If target  $i$  and context  $j$  are separated by distance ( $d$ ), to capture the intuition that closer words provide more semantic information, GloVe adds  $1/d$  to the count
  - Immediate neighbors ( $d$ ) = 1 contribute 1.0 to the count.
  - Words two steps away ( $d$ ) = 2 contribute 0.5
  - This weighting ensures the global statistics prioritize local semantic relevance over distant, potentially unrelated tokens.
- **Result:** This creates a comprehensive, weighted statistical profile of the entire corpus before training even begins.



# GloVe: How Global learns and Operates: Step by Step

## Step 2: Extracting Meaning via Probability Ratios

GloVe’s core insight is that word meaning is **not** found in the raw co-occurrence counts, but in the **ratio** of co-occurrence probabilities. This step acts as a filter to remove statistical noise.

**The Semantic Filter:** Consider two target words (*i* = ice and *j* = steam) and how they relate to various ‘probe’ words *k*:

- **Large Ratio (>> 1):** If *k* = *solid*, the ratio  $P(k|ice)/P(k|steam)$  is very high (~8.9). This identifies "solid" as a specific property of ice.
- **Small Ratio (<<1):** If *k* = *gas*, the ratio is very low (~0.085). This identifies "gas" as a specific property of steam.
- **Unitary Ratio (~1):** Mathematically cancel out words that are either related to both (water) or related to neither (fashion). This ensures the vector space focuses only on discriminative features

| X     | Solid | Gas | Water | Fashion |
|-------|-------|-----|-------|---------|
| Ice   | 19    | 6   | 54    | 1       |
| Steam | 2     | 78  | 43    | 0       |

$X_{ij}$ : Counts of word *j* in context of word *i*

$X_{ij}$  = raw co-occurrence count

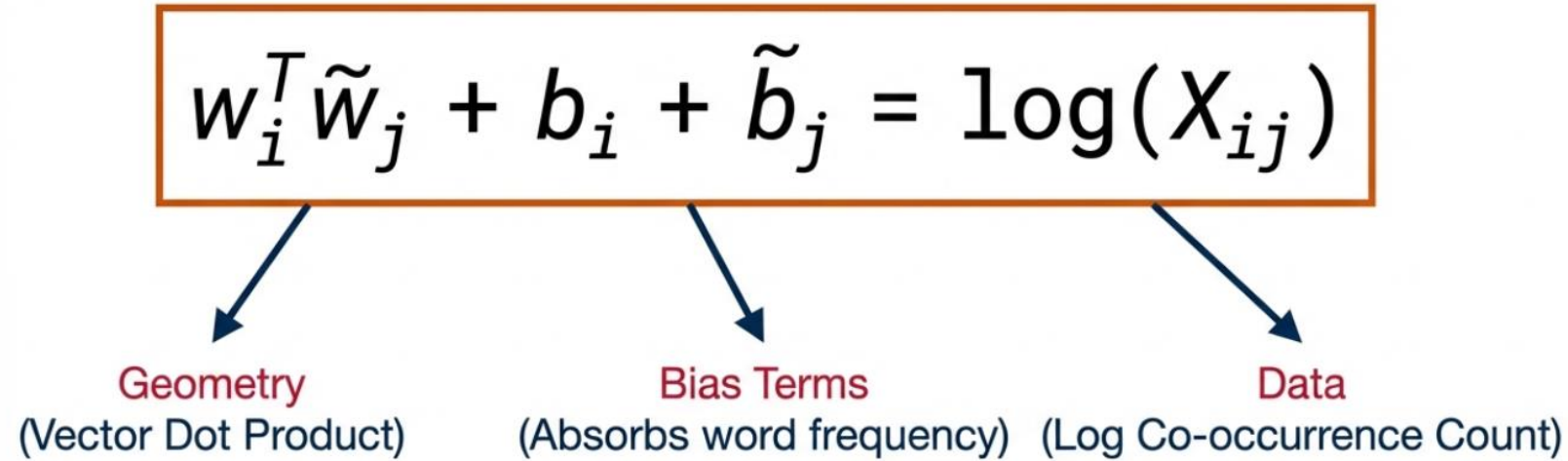
$X_i = \sum_k X_{ik}$  (Total occurrences of word *i*)

$P_{ij} = X_{ij} / X_i$  (Probability)

| Probe Word ( <i>k</i> ) | $P(k   Ice)$         | $P(k   Steam)$       | Ratio<br>$P(k Ice) / P(k Steam)$  |
|-------------------------|----------------------|----------------------|-----------------------------------|
| Solid                   | $1.9 \times 10^{-4}$ | $2.2 \times 10^{-5}$ | 8.9 (Large)<br>Related to Ice     |
| Gas                     | $6.6 \times 10^{-5}$ | $7.8 \times 10^{-4}$ | 0.085 (Small)<br>Related to Steam |
| Water                   | $3.0 \times 10^{-3}$ | $2.2 \times 10^{-3}$ | 1.36 (~1)<br>Related to both      |
| Fashion                 | $1.7 \times 10^{-5}$ | $1.8 \times 10^{-5}$ | 0.96 (~1)<br>Related to neither   |

# GloVe: Co-occurrence and Word Relationships (e.g. from the Paper)

Convert the ratios into embeddings using the equation:

$$w_i^T \tilde{w}_j + b_i + \tilde{b}_j = \log(X_{ij})$$


Geometry  
(Vector Dot Product)

Bias Terms  
(Absorbs word frequency)

Data  
(Log Co-occurrence Count)

Where:

$w_i^T \tilde{w}_k$  : Encodes the semantic relationship between the target word  $i$  and the context word  $k$

$b_i$ : Bias term that adjusts for the overall frequency of the target word  $i$ .

$\tilde{b}_k$ : Bias term that adjusts for the overall frequency of the context word  $k$ .

- Bias terms normalize the embeddings to prevent frequent words like “the” and “and” dominating relationships
- Local Information:  $w_i^T \tilde{w}_k$  captures the immediate relationship between  $i$  and  $k$  (e.g., “solid” is semantically close to “ice”)
- Global Information: The logarithm  $\log(X_{ik})$  ensures embeddings reflect global patterns (e.g., “solid” is unrelated to “steam”)

# **Appendix**

# GloVe: Co-occurrence and Word Relationships

A count matrix records how often each context word appears within a window around each target word. For simplicity, let's consider a context window of size 2. This means we look at the two words to the left and right of each target word.

| Target Word | solid | gas | water | steam |
|-------------|-------|-----|-------|-------|
| ice         | 2     | 0   | 2     | 1     |
| steam       | 0     | 1   | 1     | 0     |

From the count matrix, calculate the probability of each context word given the target word

$$P(k/i) = \frac{X_{ik}}{\sum_k X_{ik}}$$

Where:

$X_{ik}$ : Count of how often context word k appears with the target word i

$\sum_k X_{ik}$ : Total counts for target word i

Probability Matrix:

| Target Word | solid   | gas     | water   | steam   |
|-------------|---------|---------|---------|---------|
| ice         | 2/5=0.4 | 0/5=0   | 2/5=0.4 | 1/5=0.2 |
| steam       | 0/2=0   | 1/2=0.5 | 1/2=0.5 | 0/2=0   |

Ratio Matrix: GloVe focuses on ratios of co-occurrence probabilities

| Context Word | P(k ice) | P(k steam) | Ratio {P(k ice) / {P(k steam)}} |
|--------------|----------|------------|---------------------------------|
| solid        | 0.4      | 0          | Infinity (irrelevant)           |
| gas          | 0        | 0.5        | 0                               |
| water        | 0.4      | 0.5        | 0.8                             |
| steam        | 0.2      | 0          | Infinity (irrelevant)           |

# GloVe: Co-occurrence and Word Relationships (e.g. from the Paper)

Words: words “ice”, “steam”, and their co-occurrence with various context words such as “solid”, “gas”, “water”, and “fashion”.

Probability Matrix:

| Target Word | solid    | gas      | water  | fashion  |
|-------------|----------|----------|--------|----------|
| ice         | 0.00019  | 0.000066 | 0.003  | 0.000017 |
| steam       | 0.000022 | 0.00078  | 0.0022 | 0.000018 |

The raw probabilities tell us something about how often “solid” appears with “ice” or “gas” with “steam”. But these raw values are influenced by:

- 1. The frequency of the context word (e.g., common words like “water” might co-occur with both targets).
- 2. The global corpus statistics, which might not highlight unique relationships.

Instead, GloVe focuses on the ratio of probabilities:  $\frac{P(k|ice)}{P(k|steam)}$

Ratio Matrix:

| Context Word | P(k ice) | P(k steam) | Ratio {P(k ice) / {P(k steam)}} | Comments                                                                                |
|--------------|----------|------------|---------------------------------|-----------------------------------------------------------------------------------------|
| solid        | 0.00019  | 0.000022   | 8.9                             | Large value indicates “solid” is strongly associated with “ice” and weakly with “steam” |
| gas          | 0.000066 | 0.00078    | 0.085                           | Small value suggests “gas” relates more to “steam” than “ice”                           |
| water        | 0.003    | 0.0022     | 1.36                            | A ration close to 1 indicates “water” is equally related to both “ice” & “steam”        |
| fashion      | 0.000017 | 0.000018   | 0.96                            | Near Unity indicates context word is largely irrelevant to both targets                 |

# GloVe: Weighted Least Squares

- GloVe uses word-word co-occurrence matrix,  $X$ , upon which it builds its understanding of word relationships
- GloVe analyzes the ratio of co-occurrence probabilities to examine relationships between words
- GloVe uses weighted least squares regression model to learn word vectors, minimizing the difference between vector dot product and log co-occurrence counts

We saw how co-occurrence ratios reveal relationships between words. But not all co-occurrences are equally reliable:

- Frequent words (e.g., “the”): Appear everywhere, so their contribution should be limited.
- Rare words (e.g., “fashion”): May co-occur by chance, introducing noise, so their contribution should be reduced.
- Mid-frequency words (e.g., “solid”): Provide the most meaningful relationships, so they should matter the most.

## When does WLS come in?

- After converting co-occurrence counts into ratios, GloVe uses WLS during embedding training:

$$J = \sum_{i,j=1}^V f(X_{ij}) \left( w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log(X_{ij}) \right)^2$$

Where:

- $w_i^T \tilde{w}_j + b_i + \tilde{b}_j$  : Measures how well the model predicts relationship between words  $i$  &  $j$
- $\log(X_{ij})$  : The observed co-occurrence between words  $i$  &  $j$
- $w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log(X_{ij})$  : The difference therefore represents the **Error**
- $f(X_{ij})$ : Adjusts how much each co-occurrence count ( $X_{ij}$ ) contributes to the total error ~ mid frequency counts are prioritized while rare and frequent counts are down-weighted
- Square ensures the large deviations (e.g.: when the model fails to predict strong relationship) are penalized more